

ระบบบริหารการจองพื้นที่และเข้าใช้สำนักงาน

กฤษณะ พิพัฒน์พฤติกุล^{*1} และ รศ.ดร.สุรีย์ พูนิลกุล²

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี 126 ถนนประชาธิปไตย แขวงบางมด เขตทุ่งครุ กรุงเทพฯ 10140

บทคัดย่อ

บทความนี้นำเสนอการพัฒนาระบบบริหารการจองพื้นที่และเข้าใช้สำนักงาน หรือ CoDesk เพื่อสนับสนุนการบริหารจัดการการเข้าใช้ออฟฟิศขององค์กรให้มีความเป็นระบบมากขึ้น จากปัญหาการจัดการข้อมูลการจองผ่านหลายช่องทางซึ่งอาจก่อให้เกิดการจองซ้ำ ความคลาดเคลื่อนของข้อมูล การควบคุมจำนวนผู้เข้าใช้งานไม่สอดคล้องกับความต้องการของแต่ละฝ่าย และการเข้าถึงข้อมูลเกินขอบเขตของผู้ใช้งานแต่ละกลุ่ม ระบบถูกพัฒนาในรูปแบบ Web Application โดยใช้ React และ TypeScript สำหรับส่วนติดต่อผู้ใช้งาน ใช้ ASP.NET สำหรับประมวลผลตรรกะการทำงานของระบบ และใช้ Supabase PostgreSQL เป็นฐานข้อมูลหลักสำหรับจัดเก็บข้อมูลเชิงสัมพันธ์และสนับสนุนการควบคุมการเข้าถึงข้อมูล จากผลการพัฒนาพบว่า ระบบสามารถรองรับการสร้าง แก้ไข และยกเลิกรายการจอง ตรวจสอบและป้องกันการจองซ้ำ แจ้งเตือนเมื่อวันที่จองตรงกับวันหยุด ควบคุมจำนวนผู้จองตามความจุของแต่ละฝ่าย และกำหนดการมองเห็นข้อมูลตามบทบาทและฝ่ายงานของผู้ใช้งาน นอกจากนี้ ข้อมูลการจองยังสามารถนำมาสรุปเป็นรายงานเพื่อใช้ตรวจสอบจำนวนผู้จองและการใช้ความจุของแต่ละฝ่ายได้ ส่งผลให้ข้อมูลการเข้าใช้ออฟฟิศถูกจัดเก็บและตรวจสอบภายใต้กติกาเดียวกัน ลดความคลาดเคลื่อนในการจัดการข้อมูล และสนับสนุนการนำข้อมูลไปใช้ประกอบการบริหารจัดการพื้นที่สำนักงานได้อย่างเป็นระบบมากขึ้น

คำสำคัญ: การควบคุมความจุ, การควบคุมสิทธิ์ตามบทบาท, การจองเข้าออฟฟิศ, ฐานข้อมูลเชิงสัมพันธ์, โพสต์เกรสควแอล, เว็บแอปพลิเคชัน

^{*} Author's Email: 67130701930@ad.sit.kmutt.ac.th

¹ นักศึกษาปริญญาโท สาขาวิชาการบริหารระบบสารสนเทศทางธุรกิจดิจิทัล คณะเทคโนโลยีสารสนเทศ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

² อาจารย์/ผู้ช่วยศาสตราจารย์/รองศาสตราจารย์ คณะเทคโนโลยีสารสนเทศ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

บทที่ 1 บทนำ

ปัจจุบันหลายองค์กรมีการปรับรูปแบบการทำงาน สำนักงานร่วมกัน ส่งผลให้การบริหารจัดการที่เน้นเข้าออฟฟิศกลายเป็นประเด็นสำคัญในการจัดสรรทรัพยากรและการจัดการข้อมูล หากไม่มีระบบที่เหมาะสม อาจเกิดปัญหาการจองซ้ำ การใช้ทรัพยากรไม่สอดคล้องกับความต้องการของแต่ละฝ่ายและความไม่ชัดเจนในการเข้าถึงข้อมูลของผู้ใช้งานแต่ละกลุ่ม กระบวนการจองที่เน้นแบบเดิมมักอาศัยการบันทึกข้อมูลผ่านเอกสารตารางข้อมูลหรือการสื่อสารผ่านเครื่องมือทั่วไปซึ่งไม่ได้ถูกออกแบบมาเพื่อควบคุมกติกาการจองโดยตรงส่งผลให้เกิดปัญหาการจองซ้ำในระยะเวลาเดียวกัน การตรวจสอบจำนวนผู้ใช้งานในแต่ละฝ่ายทำได้ไม่แม่นยำและข้อมูลที่ได้อาจไม่อยู่ในรูปแบบเดียวกัน ทำให้ติดตาม ตรวจสอบและสรุปผลได้ยาก นอกจากนี้ ยังขาดการกำหนดสิทธิ์การเข้าถึงข้อมูลที่ชัดเจนตามสิทธิ์ของผู้ใช้ เช่น พนักงานทั่วไป ฝ่ายทรัพยากรบุคคล และผู้ดูแลระบบ จึงอาจก่อให้เกิดความเสี่ยงทั้งด้านความถูกต้องของข้อมูลและการเข้าถึงข้อมูลเกินขอบเขต

จากปัญหาดังกล่าวจึงมีแนวคิดในการพัฒนาระบบ **CoDesk** ซึ่งเป็นระบบเว็บสำหรับจองที่นั่งเข้าออฟฟิศเพื่อช่วยให้การจัดการข้อมูลการจองเป็นระบบมากขึ้นลดข้อผิดพลาดจากการบันทึกข้อมูลหลายช่องทาง และสนับสนุนการใช้งานที่สอดคล้องกับสิทธิ์ของผู้ใช้ในองค์กร ระบบนี้มุ่งเน้นการจัดเก็บข้อมูลอย่างเป็นระบบ การกำหนดสิทธิ์การเข้าถึงข้อมูลและการรองรับการใช้งานข้อมูลเพื่อการบริหารจัดการ

โครงการนี้มีวัตถุประสงค์เพื่อพัฒนาระบบจองที่นั่งเข้าออฟฟิศที่สามารถรองรับการใช้งานจริงในองค์กร โดยช่วยลดปัญหาการจองซ้ำ ควบคุมจำนวนผู้จองให้สอดคล้องกับความต้องการของแต่ละฝ่ายและกำหนดสิทธิ์การใช้งานให้เหมาะสมกับสิทธิ์ของผู้ใช้แต่ละกลุ่ม ขอบเขตของระบบครอบคลุมการจองที่นั่ง ปฏิทินการใช้งาน การจัดการฝ่ายงาน การจัดการพนักงาน การจัดการวันหยุด ระบายงานและระบบผู้ใช้กับสิทธิ์การใช้งานโดยกำหนด

สิทธิ์หลักของผู้ใช้เป็นพนักงานทั่วไป, ฝ่ายบุคคลและผู้ดูแลระบบ ซึ่งมีขอบเขตการใช้งานแตกต่างกันตามหน้าที่ ทั้งนี้ระบบรองรับการจองทั้งแบบรายวันและแบบช่วงเวลา มีการแจ้งเตือนกรณีจองตรงกับวันหยุดและกำหนดมาตรฐานวันที่พร้อมใช้เขตเวลากรุงเทพเพื่อให้การจัดเก็บและแสดงผลข้อมูลเป็นมาตรฐานเดียวกัน และสามารถนำข้อมูลไปใช้ประโยชน์ในการบริหารจัดการต่อไป

ผลที่คาดว่าจะได้รับจากโครงการนี้ คือองค์กรสามารถบริหารการเข้าใช้ออฟฟิศได้อย่างเป็นระบบมากขึ้น ลดปัญหาการจองซ้ำ ลดความคลาดเคลื่อนของข้อมูล และควบคุมการใช้ทรัพยากรให้สอดคล้องกับความต้องการของแต่ละฝ่ายได้อย่างชัดเจน ผู้ใช้งานแต่ละRole สามารถปฏิบัติงานได้ตามสิทธิ์ที่กำหนดช่วยลดความเสี่ยงจากการเข้าถึงข้อมูลเกินขอบเขตและเพิ่มความโปร่งใสในการติดตามข้อมูลการใช้งาน

บทที่ 2 ทฤษฎีและหลักการที่เกี่ยวข้อง

2.1 แนวคิดระบบฐานข้อมูลเชิงสัมพันธ์และความถูกต้องของข้อมูล

โครงการ CoDesk อาศัยแนวคิดของระบบฐานข้อมูลเชิงสัมพันธ์ในการจัดเก็บและเชื่อมโยงข้อมูลที่เกี่ยวข้องกับการจองที่นั่งเข้าออฟฟิศ เช่น ข้อมูลผู้ใช้ ฝ่ายงาน การจอง วันหยุด และสิทธิ์การใช้งานโดยแนวคิดนี้ช่วยให้สามารถกำหนดความสัมพันธ์ของข้อมูลได้อย่างชัดเจนลดความซ้ำซ้อนของข้อมูลและรองรับการกำหนดข้อบังคับของข้อมูลเพื่อให้ข้อมูลมีความถูกต้องและตรวจสอบได้ เช่น การกำหนดคีย์หลัก (Primary Key) เพื่อระบุเอกลักษณ์ของข้อมูล การกำหนดคีย์ต่างประเทศ (Foreign Key) เพื่อเชื่อมโยงความสัมพันธ์ระหว่างตาราง และการกำหนดข้อบังคับเพิ่มเติม เช่น NOT NULL, UNIQUE และ CHECK เพื่อควบคุมค่าข้อมูลให้เป็นไปตามเงื่อนไขที่ระบบกำหนด [1], [2] สำหรับโครงการนี้ เลือกใช้ Supabase PostgreSQL เป็นฐานข้อมูลหลัก เนื่องจากรองรับการจัดเก็บข้อมูลเชิงสัมพันธ์และสามารถทำงานร่วมกับระบบยืนยันตัวตนและการควบคุมสิทธิ์ใน

สภาพแวดล้อมเดียวกันได้ [2], [4] นอกจากนี้การออกแบบฐานข้อมูลของระบบยังต้องคำนึงถึงกติกาทางธุรกิจที่สำคัญ เช่น การป้องกันการจองซ้ำในช่วงเวลาเดียวกัน การควบคุมจำนวนผู้จองให้สอดคล้องกับความจริงของแต่ละฝ่าย และการรองรับการตรวจสอบข้อมูลย้อนหลัง ซึ่งเป็นองค์ประกอบสำคัญที่ทำให้ข้อมูลมีความถูกต้องและพร้อมนำไปใช้ประโยชน์ต่อการบริหารจัดการ

2.2 การควบคุมสิทธิ์การเข้าถึงข้อมูล

ระบบ CoDesk มีผู้ใช้งานหลายบทบาท ได้แก่ พนักงานทั่วไป, พนักงานฝ่ายบุคคล และผู้ดูแลระบบ ดังนั้นการออกแบบระบบจึงต้องคำนึงถึงการกำหนดสิทธิ์การเข้าถึงข้อมูลตามบทบาทของผู้ใช้ แนวคิดที่นำมาใช้คือ Role-Based Access Control (RBAC) ซึ่งเป็นแนวทางในการกำหนดสิทธิ์ตามหน้าที่ของผู้ใช้แต่ละกลุ่ม [3] ขณะเดียวกันในระดับฐานข้อมูลยังใช้แนวคิด Row-Level Security (RLS) เพื่อควบคุมการเข้าถึงข้อมูลในระดับแถว ทำให้ระบบสามารถจำกัดการมองเห็นและการแก้ไขข้อมูลได้เหมาะสมกับสิทธิ์ของแต่ละสิทธิ์ [4] การใช้ RBAC ร่วมกับ RLS ช่วยให้ระบบมีความปลอดภัยมากขึ้นและลดความเสี่ยงจากการเข้าถึงข้อมูลเกินขอบเขตโดยเฉพาะในระบบที่มีทั้งข้อมูลธุรกรรม ข้อมูลผู้ใช้และข้อมูลเพื่อการรายงาน

2.3 เทคโนโลยีที่ใช้พัฒนาระบบ

ในส่วนของการพัฒนาระบบหน้าเว็บ โครงการนี้ใช้ React สำหรับสร้างส่วนติดต่อผู้ใช้ เนื่องจากเหมาะกับการพัฒนาเว็บแอปพลิเคชันที่มีการโต้ตอบกับผู้ใช้หลายรูปแบบและช่วยจัดการโครงสร้างของหน้าจอในลักษณะ component-based ได้อย่างมีประสิทธิภาพ [5] นอกจากนี้ยังใช้ TypeScript เพื่อช่วยกำหนดชนิดข้อมูลให้ชัดเจนขึ้นในการพัฒนาหน้าจอซึ่งช่วยลดข้อผิดพลาดจากการส่งข้อมูลระหว่างส่วนต่างๆ ของระบบ สำหรับส่วนการประมวลผลฝั่งหลังบ้าน เลือกใช้ ASP.NET เพื่อรองรับตรรกะการทำงานของระบบ การตรวจสอบเงื่อนไขทางธุรกิจ และการเชื่อมต่อกับฐานข้อมูล [6] เมื่อทำงาน

ร่วมกับ Supabase PostgreSQL และ Supabase Auth จึงทำให้ระบบสามารถรองรับทั้งการจัดเก็บข้อมูล การยืนยันตัวตนและการควบคุมสิทธิ์ได้อย่างเหมาะสมกับลักษณะของโครงการ

2.4 การเปรียบเทียบเครื่องมือฐานข้อมูลที่ใช้กับ Oracle

แม้โครงการนี้ไม่ได้ใช้ Oracle Database เป็นฐานข้อมูลหลักแต่ยังคงอาศัยหลักการเดียวกันของระบบฐานข้อมูลเชิงสัมพันธ์ เช่น การจัดเก็บข้อมูลในรูปแบบตาราง การใช้ภาษา SQL การกำหนดความสัมพันธ์ของข้อมูลและการกำหนดข้อบังคับเพื่อรักษาความถูกต้องของข้อมูล [1], [7] ความแตกต่างอยู่ที่ลักษณะของแพลตฟอร์มและบริบทการนำไปใช้งานโดย Oracle มักถูกใช้ในระบบองค์กรขนาดใหญ่ที่มีเครื่องมือและสภาพแวดล้อมเฉพาะทางของตนเอง ขณะที่ Supabase PostgreSQL เหมาะกับโครงการลักษณะเว็บแอปพลิเคชันที่ต้องการความยืดหยุ่นในการพัฒนา การเชื่อมต่อกับระบบยืนยันตัวตนและการใช้งานร่วมกับเครื่องมือพัฒนาสมัยใหม่ [2], [4] ดังนั้นการเลือกใช้ Supabase PostgreSQL ในโครงการนี้จึงเหมาะสมกับขอบเขตของระบบ CoDesk และยังคงยึดหลักการสำคัญของงานออกแบบฐานข้อมูลเชิงสัมพันธ์เช่นเดียวกับที่ใช้ใน Oracle

บทที่ 3 วิธีดำเนินการ

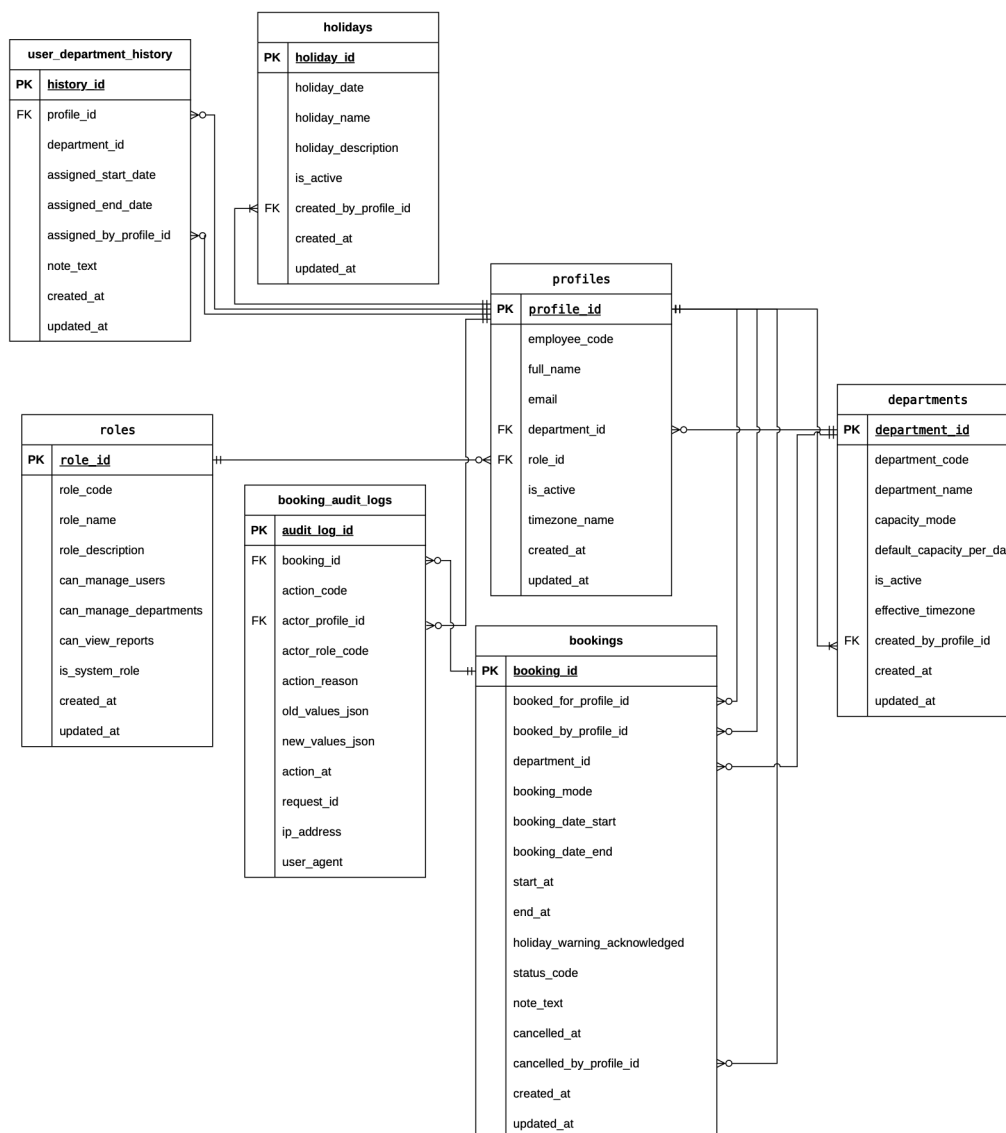
3.1 ความต้องการเชิงระบบ

ระบบ CoDesk ต้องรองรับการจองที่นั่งเข้าออฟฟิศผ่านเว็บแอปพลิเคชันโดยผู้ใช้สามารถจองได้ทั้งแบบรายวันและแบบช่วงวันหรือช่วงเวลา สามารถจองข้ามวัน จองย้อนหลังและจองล่วงหน้าได้ ระบบต้องตรวจสอบไม่ให้ผู้ใช้รายเดิมจองซ้อนในช่วงเวลาเดียวกัน และต้องตรวจสอบจำนวนผู้จองให้สอดคล้องกับความจริงของแต่ละฝ่ายตามเงื่อนไขที่กำหนดไว้ นอกจากนี้ระบบต้องรองรับการแก้ไขและยกเลิกการจองตามสิทธิ์ของผู้ใช้ และรองรับการแจ้งเตือนเมื่อวันที่จองตรงกับวันหยุด โดยยังสามารถให้ผู้ใช้ยืนยันการจองต่อไปได้

ในการใช้งานข้อมูล ระบบต้องแสดงปฏิทินการใช้งานเพื่อให้ผู้ใช้เห็นรายการจองในแต่ละวันได้และต้องกำหนดการมองเห็นข้อมูลตามบทบาทและฝ่ายงาน เช่น employee และ hr เห็นข้อมูลของตนเองและคนในฝ่ายเดียวกัน ส่วน admin สามารถเห็นข้อมูลได้ทุกฝ่าย ระบบยังต้องรองรับการจัดการฝ่ายงาน การจัดการพนักงาน การจัดการวันหยุด การจัดการผู้ใช้และสิทธิ์การใช้งาน รวมถึงการสร้างรายงานในหลายมุมมองเพื่อใช้ในการบริหารจัดการ ทั้งนี้ ระบบกำหนดให้ใช้รูปแบบวันที่ YYYY-MM-DD และใช้เขตเวลา Asia/Bangkok เป็นมาตรฐานเดียวกันทั้งระบบ

3.3 การออกแบบโครงสร้างข้อมูลและแผนภาพความสัมพันธ์ข้อมูล

การออกแบบฐานข้อมูลของระบบใช้ schema co_desk เป็นศูนย์กลางโดยกำหนดตารางหลักเพื่อรองรับข้อมูลสำคัญของระบบได้แก่ roles, departments, profiles, holidays, bookings, booking_audit_logs และ user_department_history ซึ่งครอบคลุมทั้งข้อมูลผู้ใช้ ข้อมูลฝ่ายงาน ข้อมูลการจอง ข้อมูลวันหยุด และข้อมูลสำหรับการตรวจสอบย้อนหลัง การกำหนดตารางเหล่านี้ทำให้สามารถแยกหน้าที่ของข้อมูลแต่ละส่วนได้อย่างชัดเจน และช่วยให้รองรับกติกาทางธุรกิจของระบบได้เป็นลำดับมากขึ้น



รูปที่ 3.1 แผนภาพความสัมพันธ์ข้อมูลของระบบ CoDesk

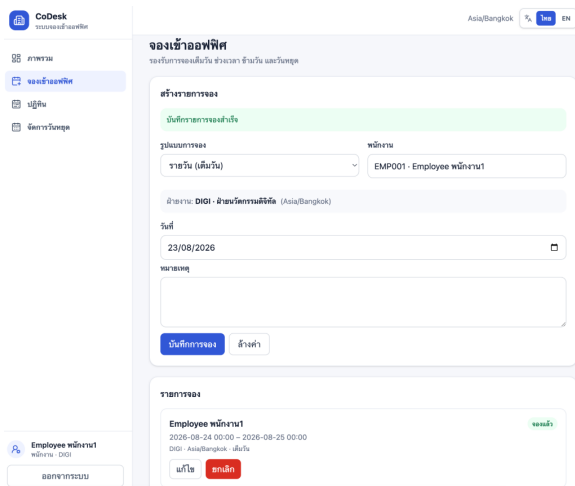
- **roles และ profiles** คือ 1 roles ต้องสามารถถูกกำหนดให้กับ profiles ได้มากกว่า 1 profile โดยแต่ละ profile จะมี role หลักได้เพียง 1 role และในบาง role อาจยังไม่มีผู้ใช้ก็ได้ แต่ profiles ทุก profiles ต้องมี role เดียว เสมอ
- **departments และ profiles** คือ 1 departments สามารถถูกกำหนดให้กับ profiles ได้มากกว่า 1 profile โดยแต่ละ profiles จะสังกัดได้เพียง 1 departments และในบาง departments อาจยังไม่มี profiles แต่ 1 profile ต้องสังกัด department เดียวเสมอ
- **profiles และ bookings (booked_for_profile_id)** คือ 1 profiles สามารถมี bookings ได้มากกว่า 1 bookings โดยแต่ละ bookings จะอ้างอิง profiles ที่ถูกจองได้เพียง 1 profiles และในบาง profiles อาจยังไม่มีรายการจองก็ได้ แต่ทุก bookings ต้องอ้างอิง profiles 1 profiles เสมอ
- **profiles และ bookings (booked_by_profile_id)** คือ 1 profiles สามารถเป็นผู้ทำรายการใน bookings ได้มากกว่า 1 bookings โดยแต่ละ bookings จะอ้างอิงผู้ทำรายการได้เพียง 1 profiles และในบาง profiles อาจยังไม่เคยทำรายการจองก็ได้ แต่ทุก bookings ต้องอ้างอิง profiles 1 profiles เสมอ
- **profiles และ bookings (cancelled_by_profile_id)** คือ 1 profiles สามารถเป็นผู้ยกเลิกรายการใน bookings ได้มากกว่า 1 bookings โดยแต่ละ bookings จะอ้างอิงได้ไม่เกิน 1 profiles และในบาง profiles อาจยังไม่เคยยกเลิกการจองก็ได้ อีกทั้ง bookings บาง bookings อาจยังไม่มีผู้ยกเลิกก็ได้ เพราะยังไม่ถูกยกเลิก
- **departments และ bookings** คือ 1 departments สามารถมี bookings ได้มากกว่า 1 bookings โดยแต่ละ bookings จะอ้างอิงได้เพียง 1 departments และในบาง departments อาจยังไม่มีรายการจองก็ได้ แต่ bookings ทุก bookings ต้องอ้างอิง departments เดียวเสมอ
- **bookings และ booking_audit_logs** คือ 1 bookings สามารถมี booking_audit_logs ได้มากกว่า 1 booking_audit_logs โดยแต่ละ booking_audit_logs จะอ้างอิงได้เพียง 1 bookings และในบาง bookings อาจยังไม่มีประวัติการเปลี่ยนแปลง แต่ทุกข้อมูลใน booking_audit_logs ต้องอ้างอิง bookings 1 bookings เสมอ
- **profiles และ booking_audit_logs (actor_profile_id)** คือ 1 profiles สามารถเป็นผู้กระทำรายการใน booking_audit_logs ได้มากกว่า 1 โดยแต่ละ booking_audit_logs จะอ้างอิง profiles ได้เพียง 1 profiles และในบาง profiles อาจยังไม่เคยมีการกระทำที่ถูกบันทึกไว้ก็ได้ แต่ทุก booking_audit_logs ต้องอ้างอิง profiles เดียว เสมอ
- **profiles และ user_department_history** คือ 1 profile สามารถมี user_department_history ได้มากกว่า 1 โดยแต่ละ user_department_history จะอ้างอิงได้เพียง 1 profiles และในบาง profiles อาจยังไม่มีประวัติการสังกัดฝ่ายก็ได้ แต่ทุก user_department_history ต้องอ้างอิง profiles เดียวเสมอ

บทที่ 4. ผลการดำเนินการ

ผลการพัฒนาระบบ CoDesk ในส่วนนี้ แสดงการทำงานที่เกี่ยวข้องกับการจองเข้าออฟฟิศ การตรวจสอบเงื่อนไขก่อนบันทึก การควบคุมความจุของฝ่าย การกำหนดสิทธิ์ในการเรียกดูข้อมูล และการสรุปข้อมูลเพื่อจัดทำรายงานโดยแสดงหน้าจอที่เกิดขึ้นจริงควบคู่กับคำสั่ง SQL ที่เกี่ยวข้องเพื่อให้เห็นว่าข้อมูลจากหน้าเว็บถูกตรวจสอบและนำไปใช้กับฐานข้อมูลอย่างไร

4.1 การบันทึกและตรวจสอบรายการจองเข้าออฟฟิศ

จากรูปที่ 4.1 ผู้ใช้งานเลือกรูปแบบการจองระบุวันที่และกดปุ่มบันทึกการจอง เมื่อข้อมูลผ่านการตรวจสอบ ระบบจะเพิ่มรายการลงฐานข้อมูลและแสดงข้อความยืนยัน พร้อมรายการจองที่สร้างขึ้นสถานะรายการ และปุ่ม สำหรับแก้ไขหรือยกเลิก



รูปที่ 4.1 การบันทึกการการจองเข้าออฟฟิศสำเร็จ

คำสั่งที่ใช้บันทึกข้อมูลแสดงในรูปที่ 4.2 โดยใช้ INSERT INTO เพื่อเพิ่มข้อมูลลงในตาราง co_desk.bookings ข้อมูลที่บันทึกประกอบด้วยพนักงานผู้จอง ผู้ดำเนินการจอง ฝ่าย รูปแบบการจอง วันที่และเวลา สถานะการรับทราบคำเตือนวันหยุด สถานะรายการ และหมายเหตุ โดยกำหนดสถานะเริ่มต้นของรายการเป็น booked หลังจากบันทึกสำเร็จ คำสั่ง RETURNING * INTO v_booking จะนำข้อมูลรายการที่เพิ่มใหม่กลับมาเก็บไว้ในตัวแปร v_booking เพื่อนำไปใช้ในขั้นตอนถัดไปของระบบ

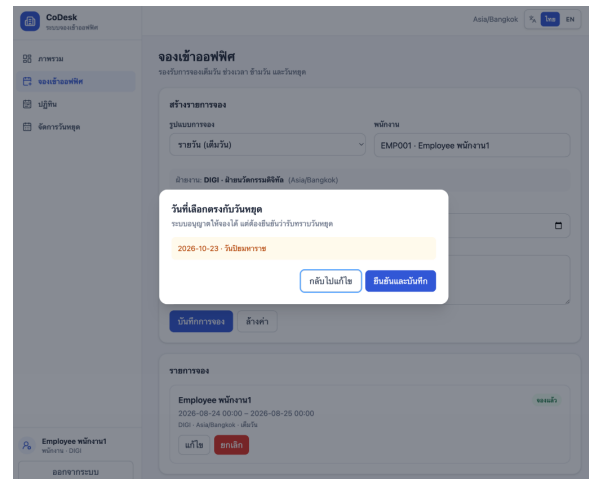
```

262 INSERT INTO co_desk.bookings (
263   booked_for_profile_id, booked_by_profile_id, department_id, booking_mode,
264   booking_date_start, booking_date_end, start_at, end_at,
265   holiday_warning_acknowledged, status_code, note_text
266 ) VALUES (
267   p_booked_for_profile_id, p_booked_by_profile_id, v_department_id, p_booking_mode,
268   v_date_start, v_date_end, v_start_at, v_end_at,
269   p_holiday_warning_acknowledged, 'booked', NULLIF(trim(p_note_text), '')
270 ) RETURNING * INTO v_booking;

```

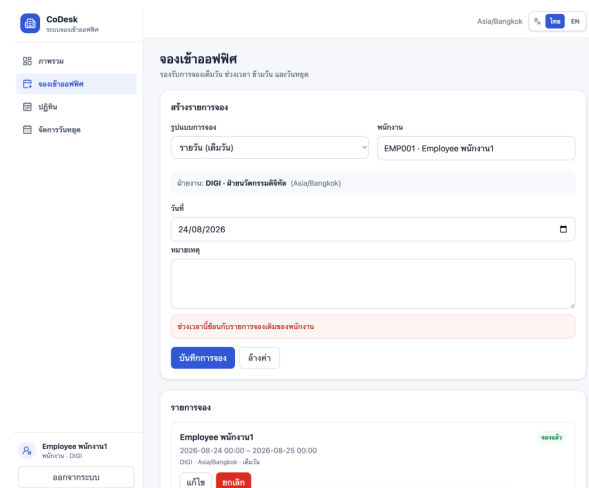
รูปที่ 4.2 คำสั่ง SQL สำหรับบันทึกข้อมูลการจองเข้าออฟฟิศ

ก่อนเพิ่มรายการลงตาราง bookings ระบบจะตรวจสอบวันที่ที่เลือกกับข้อมูลในตารางวันหยุด หากพบว่าเป็นวันหยุด ระบบจะยังไม่บันทึกรายการทันที แต่แสดงชื่อและวันที่ของวันหยุด ให้ผู้ใช้งานยืนยันก่อน ดัง รูปที่ 4.3 ผู้ใช้งานสามารถกลับไปแก้ไขวันที่หรือเลือก “ยืนยันและบันทึก” เพื่อจองในวันดังกล่าวต่อไป



รูปที่ 4.3 การแจ้งเตือนเมื่อวันที่จองตรงกับวันหยุด

นอกจากวันหยุดแล้ว ระบบจะตรวจสอบรายการเดิมของพนักงานด้วย หากช่วงเวลาที่เลือกซ้อนทับกับรายการที่มีอยู่ระบบจะไม่เพิ่มรายการใหม่ และแสดงข้อความ แจ้งเตือน ดังรูปที่ 4.4



รูปที่ 4.4 การป้องกันการจองซ้ำของพนักงาน

รูปที่ 4.5 แสดงคำสั่ง SQL สำหรับตรวจสอบรายการซ้ำ โดยค้นหาข้อมูลในตาราง co_desk.bookings ของพนักงานคนเดียวกัน เฉพาะรายการที่มีสถานะ booked และไม่นับรายการที่ถูกกำหนดให้ยกเว้น เงื่อนไข $p_start_at < b.end_at$ และ $p_end_at > b.start_at$ ใช้ตรวจสอบว่าช่วงเวลาของรายการใหม่ทับกับรายการเดิมหรือไม่ หากพบข้อมูลค่า hasConflict จะเป็นจริง และได้รับรหัสของรายการที่ซ้อนทับกลับมา

```

34 SELECT jsonb_build_object(
35     'hasConflict', EXISTS (
36         SELECT 1
37         FROM co_desk.bookings b
38         WHERE b.booked_for_profile_id = p_booked_for_profile_id
39               AND b.status_code = 'booked'
40               AND b.booking_id IS DISTINCT FROM p_exclude_booking_id
41               AND p_start_at < b.end_at
42               AND p_end_at > b.start_at
43     ),
44     'conflictingBookingId', (
45         SELECT b.booking_id
46         FROM co_desk.bookings b
47         WHERE b.booked_for_profile_id = p_booked_for_profile_id
48               AND b.status_code = 'booked'
49               AND b.booking_id IS DISTINCT FROM p_exclude_booking_id
50               AND p_start_at < b.end_at
51               AND p_end_at > b.start_at
52         ORDER BY b.start_at
53         LIMIT 1
54     )
55 );

```

รูปที่ 4.5 คำสั่ง SQL สำหรับตรวจสอบการจองซ้ำ

4.2 การควบคุมจำนวนผู้จองตามความจุของฝ่าย

ในส่วนการควบคุมความจุได้ทดสอบโดยกำหนดฝ่าย DIGI เป็นรูปแบบจำกัดและกำหนดจำนวนสูงสุดต่อวันเท่ากับ 1 คน ดังรูปที่ 4.6 เมื่อวันดังกล่าวมีผู้จองครบแล้ว และมีการเพิ่มรายการอีก ระบบจะแจ้งว่าไม่สามารถจองได้ เนื่องจากจำนวนผู้จองถึงความจุสูงสุดของฝ่าย

The screenshot shows the CoDesk interface for booking an office. The 'จองเข้าออฟฟิศ' (Office Booking) form is visible, with the department set to DIGI. The system message states: 'ไม่สามารถจองได้ เนื่องจากจำนวนผู้จอง已经达到ขีดจำกัดการจองสูงสุดแล้ว' (Cannot book because the number of bookings has reached the maximum limit).

รูปที่ 4.6 การกำหนดและควบคุมความจุของฝ่าย DIGI

คำสั่งในรูปที่ 4.7 ใช้จำนวนผู้จองเดิมของฝ่ายในวันนั้นเป็น current_count และคำนวณจำนวนหลังเพิ่มรายการใหม่เป็น projected_count หาก capacity_mode เป็น limited และ projected_count มากกว่า default_capacity_per_day ค่า exceeded จะเป็นจริง จึงไม่ดำเนินการเพิ่มรายการจองใหม่

```

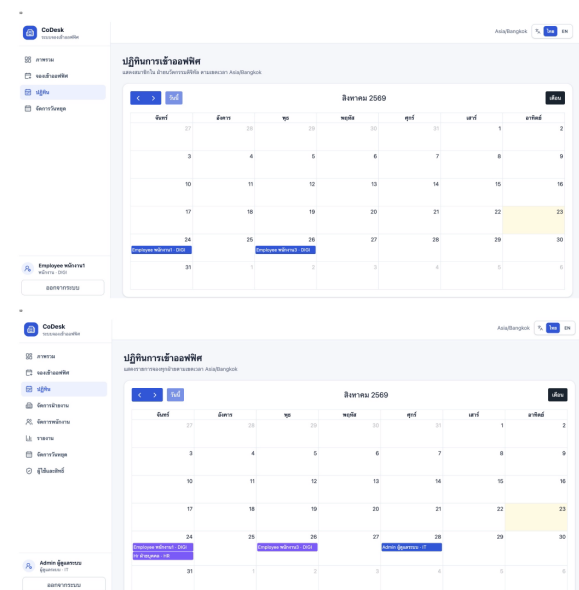
87 ), result AS (
88     SELECT daily.business_date,
89           department.capacity_mode,
90           department.default_capacity_per_day,
91           daily.current_count,
92           daily.current_count + 1 AS projected_count,
93           department.capacity_mode = 'limited'
94           AND daily.current_count + 1 > department.default_capacity_per_day AS exceeded

```

รูปที่ 4.7 คำสั่ง SQL สำหรับตรวจสอบความจุของฝ่าย

4.3 การแสดงข้อมูลตามบทบาทและฝ่ายงานของผู้ใช้งาน

ผลการทดสอบสิทธิ์การเรียกดูข้อมูลแสดงในรูปที่ 4.8 เมื่อเข้าสู่ระบบด้วยบทบาท Employee ของฝ่าย DIGI ปฏิทินจะแสดงรายการภายในฝ่าย DIGI ส่วนบทบาท Admin สามารถเห็นรายการจากทุกฝ่าย เช่น DIGI, HR และ IT ในช่วงเวลาเดียวกัน



รูปที่ 4.8 ปฏิทินการเข้าออฟฟิศตามบทบาทผู้ใช้งาน (n) Employee (ข)

Admin

การจำกัดข้อมูลในระดับฐานข้อมูลใช้ Row-Level Security (RLS) ตามรูปที่ 4.9 โดย Policy bookings_read อนุญาตให้ SELECT ข้อมูลเมื่อผู้ใช้งานเป็น Admin เป็นเจ้าของรายการจอง หรืออยู่ใน

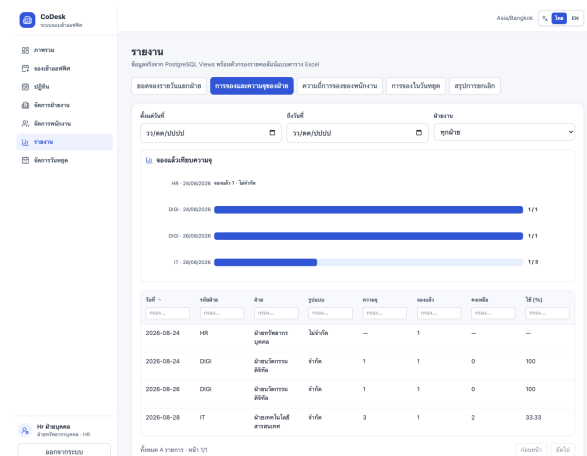
ฝ่ายเดียวกับรายการ เงื่อนไขนี้ทำให้ข้อมูลที่อ่านได้สอดคล้องกับขอบเขตของผู้ใช้งานแต่ละบทบาท

```
99 DROP POLICY IF EXISTS bookings_read ON co_desk.bookings;
100 CREATE POLICY bookings_read ON co_desk.bookings FOR SELECT USING (
101     co_desk.is_admin()
102     OR booked_for_profile_id = co_desk.current_profile_id()
103     OR department_id = co_desk.current_department_id()
104 );
```

รูปที่ 4.9 คำสั่ง SQL สำหรับกำหนดสิทธิ์การเข้าถึงข้อมูลการจอง

4.4 การนำข้อมูลไปใช้เพื่อการบริหารจัดการ

รูปที่ 4.10 แสดงรายงานการจองและความจุของฝ่าย ซึ่งสรุปข้อมูลเป็นกราฟและตาราง ประกอบด้วยวันที่ รหัสฝ่าย ชื่อฝ่าย รูปแบบความจุ ความจุสูงสุด จำนวนผู้จอง จำนวนคงเหลือ และอัตราการใช้ความจุเป็นร้อยละ ทำให้สามารถเปรียบเทียบการใช้ความจุของแต่ละฝ่ายใน ช่วงเวลาเดียวกันได้



รูปที่ 4.10 รายงานการจองและความจุของฝ่าย

ข้อมูลของรายงานที่มาจาก View vw_department_capacity_utilization ดังรูปที่ 4.11 โดยนำจำนวนการจองรายวันมา JOIN กับข้อมูลฝ่าย ค่า default_capacity_per_day ถูกใช้เป็น capacity_per_day และจำนวนรายการจองเป็น booked_count สำหรับฝ่ายแบบ limited คำสั่ง CASE จะคำนวณ remaining_capacity และ utilization_percentage ก่อนส่งข้อมูลไปแสดงในหน้ารายงาน

```
20 CREATE OR REPLACE VIEW co_desk.vw_department_capacity_utilization AS
21 SELECT
22     daily.business_date,
23     dep.department_id,
24     dep.department_code,
25     dep.department_name,
26     dep.capacity_mode,
27     dep.default_capacity_per_day AS capacity_per_day,
28     daily.active_booking_total AS booked_count,
29     CASE WHEN dep.capacity_mode = 'limited'
30     THEN GREATEST(dep.default_capacity_per_day - daily.active_booking_total, 0)
31     ELSE NULL END AS remaining_capacity,
32     CASE WHEN dep.capacity_mode = 'limited'
33     THEN ROUND((daily.active_booking_total::numeric / dep.default_capacity_per_day) * 100, 2)
34     ELSE NULL END AS utilization_percentage
35 FROM co_desk.vw_daily_department_bookings daily
36 JOIN co_desk.departments dep ON dep.department_id = daily.department_id;
```

รูปที่ 4.11 คำสั่ง SQL

สำหรับจัดเตรียมข้อมูลรายงานการใช้ความจุของฝ่าย

จากข้อมูลตัวอย่าง ฝ่าย DIGI มีความจุ 1 คนและมีผู้จอง 1 คน จึงมีการใช้ความจुर้อยละ 100 ส่วนฝ่าย IT มีความจุ 3 คนและมีผู้จอง 1 คน คิดเป็นร้อยละ 33.33 ข้อมูลดังกล่าวทำให้เห็นทั้งจำนวนที่ใช้งานแล้วและจำนวนคงเหลือของแต่ละฝ่ายจากรายงานเดียวกัน

จากผลการทดสอบข้างต้น พบว่าคำสั่ง SQL ที่ใช้ในระบบสามารถรองรับการตรวจสอบและจัดการข้อมูลการจองตามเงื่อนไขที่กำหนด และแสดงผลผ่านเว็บแอปพลิเคชันได้ตามขอบเขตของโครงการ

5. บทสรุป

5.1 สรุปผลการดำเนินงาน

โครงการ CoDesk พัฒนาขึ้นเพื่อช่วยแก้ปัญหาการจัดการข้อมูลการเข้าใช้ออฟฟิศที่อาจเกิดจากการดำเนินงานผ่านหลายช่องทาง เช่น การจองซ้ำในช่วงเวลาเดียวกัน การควบคุมจำนวนผู้เข้าใช้งานให้สอดคล้องกับความจุของแต่ละฝ่าย และการเข้าถึงข้อมูลเกินขอบเขตของผู้ใช้งานแต่ละกลุ่ม โดยพัฒนาเป็น Web Application และจัดเก็บข้อมูลในฐานข้อมูล Supabase PostgreSQL เพื่อให้ข้อมูลการจองและข้อมูลที่เกี่ยวข้องถูกจัดเก็บและตรวจสอบภายใต้โครงสร้างเดียวกัน

จากผลการดำเนินงานพบว่า ระบบสามารถตรวจสอบและป้องกันการจองซ้ำ แจ้งเตือนเมื่อวันที่จองตรงกับวันหยุด ควบคุมจำนวนผู้จองตามความจุที่กำหนดของแต่ละฝ่าย และกำหนดการมองเห็นข้อมูลตามบทบาทและฝ่ายงานของผู้ใช้งานได้ นอกจากนี้ ข้อมูลการจองยัง

สามารถนำมาสรุปเป็นรายงานเพื่อให้ฝ่ายที่เกี่ยวข้องตรวจสอบจำนวนผู้จองและการใช้ความจุของแต่ละฝ่ายได้ ผลที่ได้ช่วยลดความคลาดเคลื่อนในการจัดการข้อมูล เพิ่มความชัดเจนในการควบคุมการเข้าใช้ออฟฟิศ และทำให้ข้อมูลสามารถนำไปใช้ประกอบการบริหารจัดการได้อย่างเป็นระบบมากขึ้น

5.2 แนวทางการพัฒนาในอนาคต

แนวทางสำคัญในการพัฒนาระบบต่อไป คือ การเชื่อมต่อการเข้าสู่ระบบของ CoDesk กับ Microsoft

Entra ID (เดิม Azure Active Directory: Azure AD) ขององค์กร เพื่อให้พนักงานสามารถใช้บัญชี Microsoft ของบริษัทเข้าสู่ระบบในลักษณะ Single Sign-On (SSO) แทนการจัดการบัญชีและรหัสผ่านแยกเฉพาะสำหรับ CoDesk ซึ่งจะช่วยลดการจัดการบัญชีผู้ใช้งานซ้ำ และทำให้กระบวนการยืนยันตัวตนสอดคล้องกับระบบขององค์กร โดย Microsoft Entra ID จะทำหน้าที่ยืนยันตัวตนผู้ใช้งาน (Authentication) ขณะที่ CoDesk ยังคงกำหนดบทบาท ฝ่ายงาน และขอบเขตการเข้าถึงข้อมูลของ Employee, HR และ Admin ภายในระบบ (Authorization) ตามโครงสร้างที่พัฒนาไว้เดิม

เอกสารอ้างอิง

- [1] PostgreSQL Global Development Group, “PostgreSQL Documentation,” PostgreSQL. [Online]. Available: <https://www.postgresql.org/docs/current/>. [Accessed: Mar. 21, 2026].
- [2] Supabase, “Database Overview,” Supabase Docs. [Online]. Available: <https://supabase.com/docs/guides/database/overview>. [Accessed: Mar. 21, 2026].
- [3] National Institute of Standards and Technology, “The NIST Model for Role-Based Access Control: Toward a Unified Standard,” NIST. [Online]. Available: <https://www.nist.gov/publications/nist-model-role-based-access-control-towards-unified-standard>. [Accessed: Mar. 21, 2026].
- [4] Supabase, “Row Level Security,” Supabase Docs. [Online]. Available: <https://supabase.com/docs/guides/database/postgres/row-level-security>. [Accessed: Mar. 21, 2026].
- [5] Meta Platforms, Inc., “Learn React,” React Docs. [Online]. Available: <https://react.dev/learn>. [Accessed: Mar. 21, 2026].
- [6] Microsoft, “Create web APIs with ASP.NET Core,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/web-api/>. [Accessed: Mar. 21, 2026].
- [7] Oracle, “Oracle Database SQL Language Reference,” Oracle Help Center. [Online]. Available: <https://docs.oracle.com/en/database/oracle/oracle-database/23/sqlrf/>. [Accessed: Mar. 21, 2026].